Multi-Task Learning for Stock Price Prediction

Computational Intelligence (CIS*4780) - March 2026

Stella Rovazzi
srovazzi@uoguelph.ca

Matthew Lawford
mlawford@uoguelph.ca

Julia Horner
jhorne03@uoguelph.ca

Abstract

Insert abstract here

I. Introduction

Forecasting of financial time series is a significant challenge in computational intelligence due to the high volatility, non-linearity and inconsistent variety of global markets. Traditional univariate models often try to predict stock price movements in isolation, not accounting for the deep rooted dependencies and shared behaviours between firms in the same sector. This project investigates Multi-Task Learning, a form of inductive transfer to improve predictive accuracy by explicity modeling these interconnections.

By utilizing shared representations, the Multi-Task Learning framework forces the model to favor parameters that generalize across different but associated market behaviours. This approach leverages sector-wide knowledge to improve the performance of individual target tasks. Our study focuses on a diverse cohort of 25 companies spanning the hardware and software sectors, derived from the iShares S&P/TSX Capped Information Technology Index and Vanguard Information Technology ETFs.

The primary objective is to develop a robust architecture capable of capturing complex, non-linear dependencies in financial data from 2009 to 2017. To evaluate the efficacy of the Multi-Task Learning approach, we compare two sophisticated architectures:

- Long Short-Term Memory (LSTM) Networks: A recurrent architecture designed for sequential data, implemented with a shared hidden layers and task-specific output heads.
- Multi-Task Extra Trees: An ensemble method utilizing multi-output regression to optimize node splits across all tasks simultaneously.

By comparing these multi-tasking frameworks against traditional single-prediction baselines, this study demonstrates how shared representations can reduce variance and provide more stable price forecasts in volatile financial environments.

II. Motivation and Objectives

Motivation

Comparing and exploring multi-tasking

Objectives

The primary objective of this research is to evaluate whether a multi-task learning framework can improve the predictive accuracy of individual stock prices by leveraging historical information from correlated stocks within the same sector. To achieve this, the project also contains the following sub-objectives:

- Comparative Performance Analysis: To perform a good comparison between multi-task learning architectures and traditional single-task (univariate) baselines to determine if modeling stocks in isolation fails to capture critical sector-wide dependencies.
- Validation of Inductive Transfer: To assess the effectiveness of inductive transfer, specifically the ability to improve the performance of a target task after learning to model related tasks within the technology sector.
- Evaluation of Inductive Bias: To determine if forcing the model to favour parameters that are effective across multiple target tasks leads to more stable forecasts in highly volatile financial environments.
- Cross-Model Generalization: To apply the multi-task learning approach across 2 distinct computational intelligence architectures, Long Short-Term Memory (LSTM) networks and Multi-Task Extra Trees to ensure the findings are robust across different algorithmic types.

- Sector Interrelationship Testing: To validate if the shared representation between hardware and software firms provides a accuracy boost by modeling their interconnections rather than modeling them in isolation.

III. Methodology

1) Data Availability: Given the abundance of detailed historical stock market data, the dataset was carefully chosen. A mix of hardware and software companies between 2009 and 2017 was selected to ensure that the data exhibits behaviours uninfluenced by external severe market-changing events. Training on this data allowed for the architecture to capture the complex, non-linear market behaviors that traditional models struggle to represent. The timeframe was specifically restricted to the period from 2009 through 2017 to avoid two major economic anomalies: the 2008 financial crash and the COVID-19 pandemic. Excluding these catastrophic events prevents the model from unintentionally being trained on massive market disruptions.

2) Datasets: The specific companies in the dataset were taken from the iShares S&P/TSX Capped Information Technology Index ETF and the Vanguard Information Technology ETF. These professionally curated funds were used to ensure a baseline of market viability and relevance. To guarantee sufficient training data, a total of 25 companies were selected: 5 from iShare and 20 from the Vanguard ETF. The selected companies were subsequently divided into three categories based on their market capitalization: large-cap (greater than \$120,000,000 USD), mid-cap (between \$120,000,000 and \$50,000,000 USD), and small-cap (less than \$50,000,000 USD).

3) Intra-Sector Relationships and Multitasking: A varied composition of technology companies, spanning across a large range of valuations, was deliberately selected to optimize the architecture for multi-task learning. This introduced the models to complex intra-sector relationships, allowing them to learn how behaviours in one sector influence or correlate with another. By multi-tasking across these datasets, the model avoids overfitting to a single company size and develops a stronger understanding of the underlying patterns of the market.

A. Algorithms

LSTMs: explination To implement this, we used pytorch.

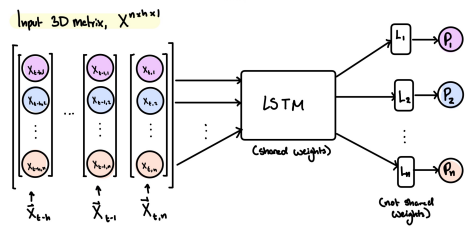

Fig. 1. Diagram of hard parameter sharing and inputs for an LSTM Multi-Tasking model. (Drawn by group).

Multi-Tasking Extra Trees (or Extremely Randomized Trees): Extra Trees (ET) is a tree ensemble method like Random Forests, but it produces more decorrelated trees. It uses the entire data set to build decision trees, rather than bootstrapping as in random forests. It also differs in its threshold selection, which randomly selects where to split the feature rather than using an optimization method. It still chooses m_{try} number of random features to choose the feature splits to compare, where $m_{try} = \lfloor \frac{p}{3} \rfloor$ or $m_{try} = \lfloor \sqrt{p} \rfloor$ and p is the number of features. There are two relevant extensions to this algorithm that make it similar to our LSTM hard parameter multi-tasking [1].

The first is introducing multi-output, which essentially means the loss function is the summation of sample error and task outputs. This effectively makes each node split consider the error of the candiate splits' effect across all tasks [1]. The second is introducing task-focused splits, or the ability to make decisions based on what the task is being predicted. To do this, a random group of m_{try} features and S random possible subsets of tasks are selected (S$\epsilon \mathbb{Z} < T$ randomly selected). Each feature split is randomized, and the multi-output error is calculated. Then, f_k (Equation 1) is calculated for each candidate task subset or feature; this is the prediction. The error is calculated from the prediction and used to choose the best split at the node [1].

$$f_k = \frac{\sum_{\nu \epsilon I_k} y_\nu + \alpha \frac{1}{\lfloor I \rfloor} \sum_{w \epsilon I} y_w}{\lfloor I_k \rfloor + \alpha} \tag{1}$$

"I denotes the set of sample indexes at the current node, I_k the set of sample indexes of the task k at the current node and α is a regularization parameter weighting the influence of the tasks with default value 1. Then, the task cut point is randomly selected from the interval $(\min_k f_k, \max_k f_k)$" - Schmid et. al [1]

To implement this method, we used R, since there is

a pre-made R package for multi-tasking tree enssemblers. It is technically archived in newer versions of R, but can be downloaded here. This was decided because the paper explaining the metholody used this package [1].

B. Metrics and Evaluation

Rolling Cross-Validation: To validate the model model, rolling cross validation was used. This method slides a set length "window" along the folds, validating with the last fold of the window. A visual for this method is shown in figure 2. It's ideal for time series data [2].

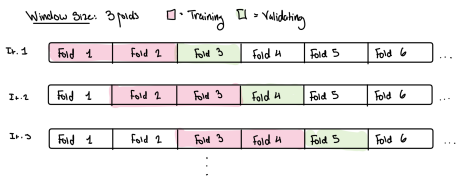

Fig. 2. Rolling cross validation explination. (Drawn by Group)

Mean Absolute Error (Criterion and Metric): Mean Absolute Error (MAE) is a metric robust to outliers [2]. For that reason, the MAE was chosen as the loss function, and the models were evaluated with this metric. The MAE could not be used as the "criterion" or error to minimize in extra trees because the R package did not support it.

Training and Evaluation: The data was split into 80-20 training and evaluation. This made the evaluation period approximately cover the years 2015 to 2017.

C. Preprocessing

Misaligned Dates: Due to different market holidays, some days in the data included both null values and recorded prices. The days with null values were removed to keep the dataset consistent. Dropping a few days of data should not obscure the stocks' growth patterns and relationships.

Logarithmic Scale: The data was scaled logarithmically. This was to ensure stable predictions by reducing effect of outliers on the model and to account for the exponential patterns in each stock's data. The predictions become more stable because most analysis techniques assume additive data, including machine learning. [3].
The scaling affect is shown in appendix A, the medium stock ACIW is on a similar scale to the large stock AAPL after logging the data.

Lag Length: Lags are data from a previous date; for example, lag 1 at time t is the data point at time $t-1$ [2]. This idea is from time series, where it is assumed that lags of the data have some affect on the current t's data.
For this model, 60 lags were used because a quarter of the fiscal year is approximately this many days [4]. This value considers only days when the markets are actually open.

LSTMs: Pytorch requires LSTM inputs to be 3D tensors [5]. The data looks at stocks, with the only attribute being the stock price. Referring to figure 1, the matrix on the left would be an example 3D tensor input, with each vector $\vec{x}$ representing all the stocks of interest at the given t. There is no third dimension depicted, as the only attribute of each stock of interest is the given price at time t.
The times within the input are $\vec{x_{t,n}}$ and the lags of $\vec{x_{tn}}$, which represent our rolling window, since $\vec{x_{t+1}}$ should use all the previous lags for its prediction.

Extra Trees: To turn the stock price data into an acceptable input for a tree ensemble, the lags must be added manually. Each datapoint will then have 60 attributes representing the 60 associated lags lags [6]. Days without all 60 previous lags have been dropped.

D. Building the Models

LSTMs: describe the things we need to tune? And how single vs mutli looks diffrent (not what we chose just what we did method wise). Why chosen hyper parameters, why others not chosen.

Extra Trees: For Extra Trees, single-task and multi-task, the parameters that could be tuned are [1]:

- The number of grown trees
- The cardinality of the sample of features at every node is m_{try}
- The number of random split values at each node
- The minimum number of observations that each terminal node should contain (stopping criteria)

The models in this paper are tuned for m_{try}, the number of random splits/cuts, and the number of terminal node observations. The number of trees was set to the default of 500 [7]. Any excess grown tree doesn't impact the predictive power of the model, and 500 is likely enough for the tasks of interest [8].

IV. Results

For the single-task LSTM, IBM and BlackBerry stocks were evaluated. The plots to tune the hyperparameters are

in CHANGE TO THE CORRECT APPENDIX. The model performance was optimized based on the lowest validation Mean Absolute Error (MAE).

Hyperparameters

BlackBerry (BB.TO): For the BlackBerry dataset, the optimal validation MAE was achieved using 42 epochs, 25 hidden nodes, a learning rate of 0.0008, and a single LSTM layer.

IBM: For the IBM dataset, the optimal validation MAE was achieved using 32 epochs, 100 hidden nodes, a learning rate of 0.0011, and a single LSTM layer.

Algorithm Performances

BlackBerry's test evaluation of MAE was 0.0371 for single task LSTM, and IBM's test evaluation of MAE was 0.0628.

A. Multi-Task LSTM

The Multi-Task LSTM architecture was designed to leverage shared temporal features across a synchronized dataset of 25 stocks. By utilizing shared hidden layers, the model implements an inductive bias that favours parameters generalizing across the technology sector.

Hyperparameters: The optimal configurations for the MTL model was determined through a systematic 3 stage ablation study evaluating layer count, hidden dimension size, and learning rates. The final architecture selected for the 200-epoch evaluation run consisted of:

- **LSTM Layers:** 2. As illustrated in Figure B1, 2 layers provided the optimal balance between model capacity and generalization, whereas deeper architectures showed a sharp increase in validation error.
- **Hidden Dimension:** 128. The selection of 128 hidden units was justified by the performance trends in Figure B2, proving sufficient to capture high-dimensional cross-stock correlations without over-parameterization.
- **Learning Rate:** 0.0005. Figure B3 highlights that this rate allowed for the most stable descent into the convergence learning cliff, avoiding the instability found in more aggressive rates.
- **Dropout:** 0.2. Applied to the shared hidden layers to provide robust regularization against the high volatility of the technology sector.
- **Batch Size:** 64. Utilized to provide a stable gradient estimate for the weighted sum of task-specific loss functions while maintaining enough stochastic noise for effective generalization.

B. Single-Task Extra Trees

For single-task extra trees, IBM and Blackberry (BB.TO) stocks were evaluated. The plots used to tune the hyperparmeter are in D.

IBM: IBM Had the smallest validation MAE with terminal node size 3 D1, 2 random cuts D3, and m_try size $p/3$ D2. IBM's evaluation MAE was 0.01502721 for single task ET. Figure 3 shows the prediction of this model over the monthly evaluation period.

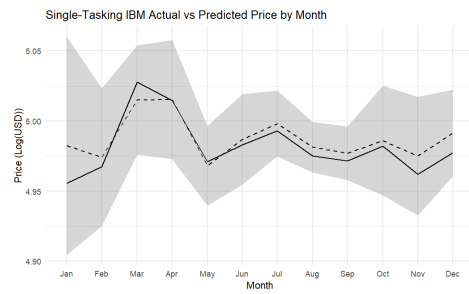

Fig. 3. Evaluation prediction of IBM stock using single-task extra tree. Solid line is true value; dotted is prediction; and shaded area is the variance.

Nearing the later part of the calendar year, the single-task model seems to over-esitmate the IBM stock. As well, the variance at the begining of the evaluation period is high.

Blackberry (BB.TO): Blackberry had the smallest validation MAE with terminal node size 3 D4, 8 random cuts D6, and m_try size $\sqrt{p}$ D5. Blackberry's evaluation MAE was 0.02308594 for single-task ET. Figure 4 shows the prediction of this model over the monthly evaluation period.

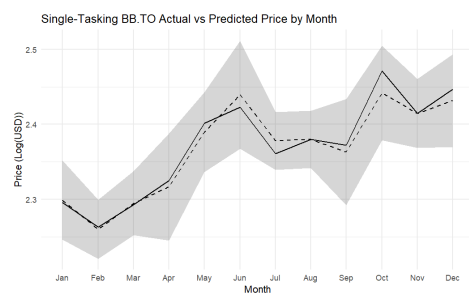

Fig. 4. Evaluation prediction of BB.TO stock using single-task extra tree. Solid line is true value; dotted is prediction; and shaded area is the variance.

The variance of the prediction is consistent throughout the evaluation period. There is a mix between over and under reporting the price of the blackbery stock.

C. Multi-Task Extra Trees

Hyperparameters: Appendix E shows the tuning plots for this section. Tuning showed 3 nodes and 5 nodes had almost

equivalent MAE, so we chose 3 nodes. Tuning also showed $p/3$ was the best for m_try and 10 random cuts would be ideal for multi-tasking.

Algorithm Evaluation Performances: Below is a table that shows the MAE of evaluation for each stock prediction in the multi-tasking extra trees model:

Task	Mean of MAE for Evaluation Window
AAPL	0.010017864
ACIW	0.013566978
AMD	0.031279329
BB.TO	0.015650922
BDC	0.017563812
BELF	0.019272468
BLKB	0.013493255
BMI	0.011254196
CLS.TO	0.012401716
CRM	0.013950118
CSCO	0.009417804
DSG.TO	0.011196125
GIB-A.TO	0.009294681
IBM	0.011959280
INTC	0.009181391
KXS.TO	0.017382097
MSFT	0.009562804
NOVT	0.013969065
NTCT	0.015411194
OLED	0.024107319
ORCL	0.008827831
OTEX.TO	0.009609518
PLAB	0.015337510
SLAB	0.014071373
UCTT	0.023878160
VSAT	0.012968538

Figures 6 and 5, visualizes the multi-tasked predictions of the stocks previously predicted with single-task models.

The IBM multi-task prediction has consistent variance throughout the evaluation period. It still, however, over estimates the cost of the stock nearing the end of the calendar year.

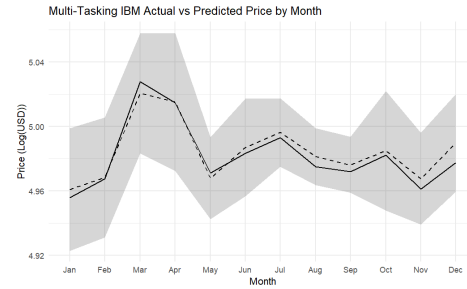

Fig. 5. Evaluation prediction of IBM stock using multi-task extra tree. Solid line is true value; dotted is prediction; and shaded area is the variance.

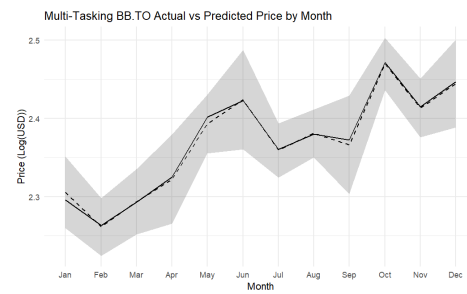

Fig. 6. Evaluation prediction of BB.TO stock using multi-task extra tree. Solid line is true value; dotted is prediction; and shaded area is the variance.

The blackberry stock prediction holds very close to the true prediction.

V. Discussion

A. Findings

B. Interpretation of Results

1) Single vs Multi:

2) LSTM MTL vs ExtraTrees MTL:

C. Advantages and Disadvantages of Multi-Tasking

VI. Conclusion

Future Works

To further this project, the remaining stocks of interest could have their single-task counterparts calculated and compared. This would provide a better understanding of how multitasking affects the predictions.

Tthe intermediate step of multi-input single-output predictions could also be explored. This would reveal the effect of shared weights in multi-tasking, because the multi-input single-output model would use the same input data as our multi-tasking model, but it would optimize for only one prediction.

REFERENCES

[1] L. Schmid, A. Gerharz, A. Groll, and M. Pauly, "Tree-based ensembles for multi-output regression: Comparing multivariate approaches with separate univariate ones," *Computational Statistics & Data Analysis*, vol. 179, p. 107628, 2023. [Online]. Available: https://www.sciencedirect.com/science/article/pii/S0167947322002080

[2] J. Slater, "Time series analysis (stat*4360)," University of Guelph, 2025.

[3] "Decomposition of time series," 2025. [Online]. Available: https://medium.com/@m.capurso/decomposition-of-time-series-071ce7b7a5a0

[4] R. Malhotra, "How many trading days in a year?" 2026. [Online]. Available: https://www.stockgro.club/blogs/trading/how-many-trading-days-in-a-year/#:~:text=Trading%20days%20don't%20spread,a%20shorter%20month%20for%20trading.

[5] "Sequence models and long short-term memory networks," 2017. [Online]. Available: https://docs.pytorch.org/tutorials/beginner/nlp/sequence_models_tutorial.html

[6] I. P. Carrascosa, "Forecasting the future with tree-based models for time series," 2025. [Online]. Available: https://machinelearningmastery.com/forecasting-the-future-with-tree-based-models-for-time-series/

[7] "extratrees: Function for training extratree classifier or regression." [Online]. Available: https://www.rdocumentation.org/packages/extraTrees/versions/1.0.5/topics/extraTrees

[8] "The effects of the depth and number of trees in a random forest," 2025. [Online]. Available: https://www.geeksforgeeks.org/machine-learning/the-effects-of-the-depth-and-number-of-trees-in-a-random-forest/

APPENDIX

A. Raw Data

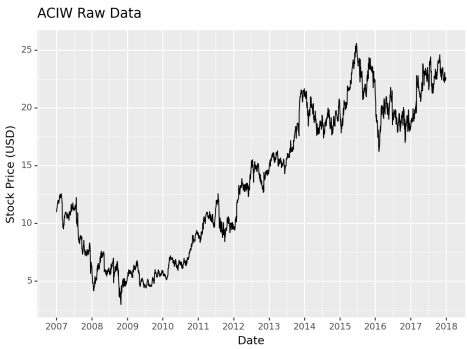

Fig. A1. Raw price data of ACIW Stock

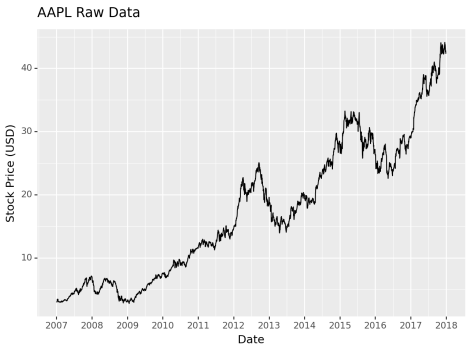

Fig. A2. Raw price data of APPL Stock

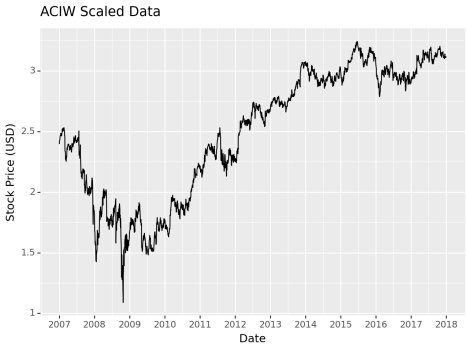

Fig. A3. Logged price data of ACIW Stock

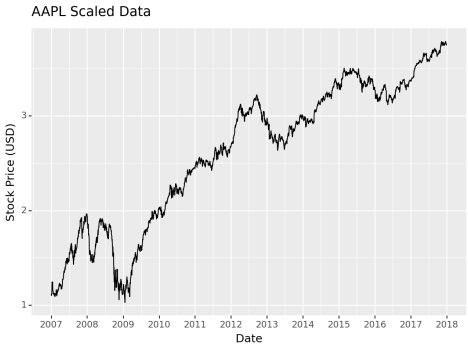

Fig. A4. Logged price data of APPL Stock

B. Multi-Task LSTM Hyperparameter Tuning

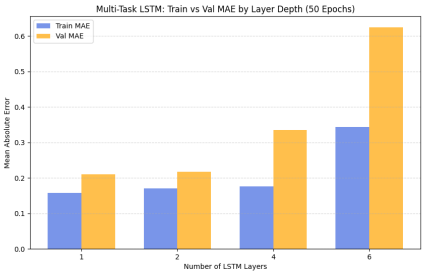

Fig. B1. Tuning Multi-Task LSTM: Number of Layers. Evaluation of 1 to 6 layers showed 2 layers provided the optimal balance of capacity and generalization.

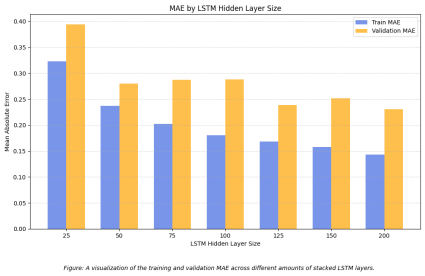

Fig. B2. Tuning Multi-Task LSTM: Hidden Dimension. Performance across a range of 25 to 200 units justified the selection of 128 hidden units.

Fig. B3. Tuning Multi-Task LSTM: Learning Rate. Comparison of rates between 0.01 and 0.0001 identified 0.0005 as the most stable for sectoral synchronization.

C. Multi-Task LSTM Training Results

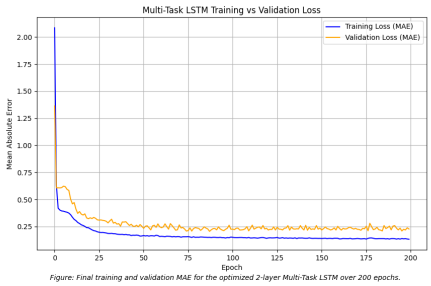

Fig. C1. Final 200-Epoch Training Curve: Multi-Task LSTM. Visualizing the convergence and stability of the optimized 2-layer, 128-hidden unit model.

D. Single-Task Extra Trees Tuning

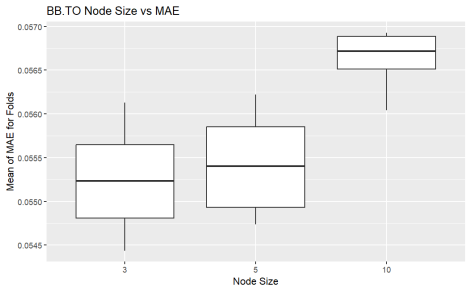

Fig. D1. Tuning Blackberry node size

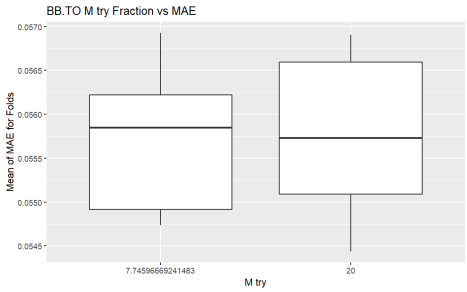

Fig. D2. Tuning Blackberry m_{try} fraction, choice between $p/3$ and $\sqrt{p}$ where p is feature length

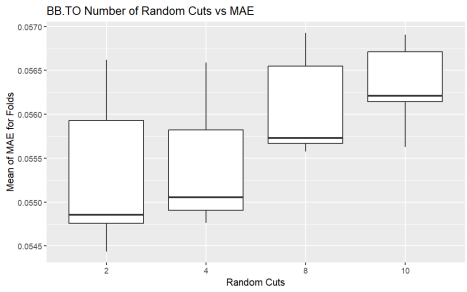

Fig. D3. Tuning Blackberry number of random cuts

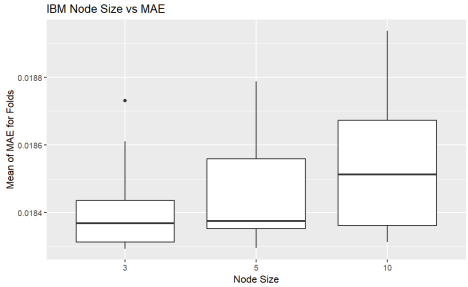

Fig. D4. Tuning IBM node size

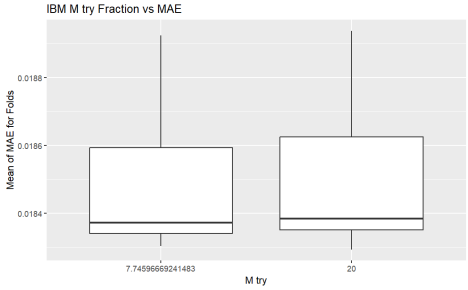

Fig. D5. Tuning IBM m_{try} fraction, choice between $p/3$ and $\sqrt{p}$ where p is feature length

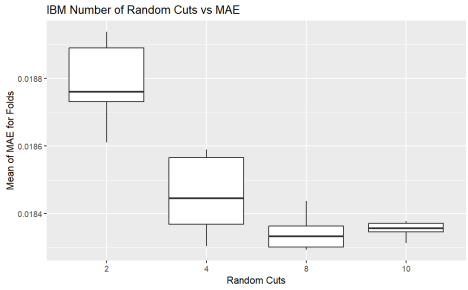

Fig. D6. Tuning IBM number of random cuts

E. Multi-Task Extra Trees Hyperparameter Tuning

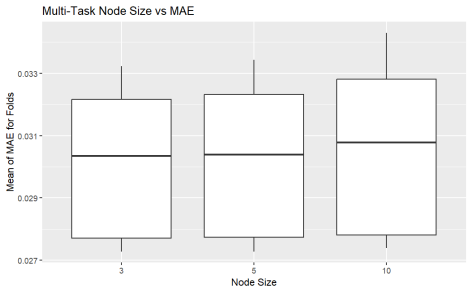

Fig. E1. Tuning Multi-Task Extra Trees node size

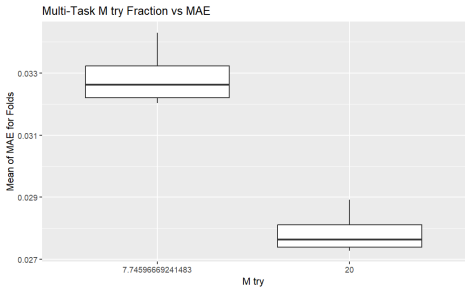

Fig. E2. Tuning Multi-Task Extra Trees m_{try} fraction, choice between $p/3$ and $\sqrt{p}$ where p is feature length

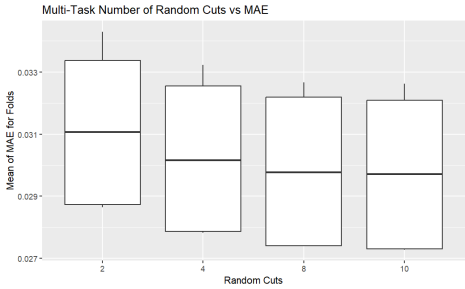

Fig. E3. Tuning Multi-Task Extra Trees number of random cuts